

Rep Smarter, Not Harder: AI Hypertrophy Coaching with Wearable Sensors and Edge Neural Networks

1st Musa Azeem

College of Engineering and Computing
University of South Carolina
Columbia, SC, USA
mmazeem@email.sc.edu

2nd Grant King

College of Engineering and Computing
University of South Carolina
Columbia, SC, USA
gking@email.sc.edu

3rd Savannah Noblitt

College of Engineering and Computing
University of South Carolina
Columbia, SC, USA
snoblitt@email.sc.edu

Abstract—abstract

I. INTRODUCTION

A. Background

Exercise science background

B. Related Work

Literature review

C. Contributions

What we did new - describe near-failure reps.

II. MATERIALS AND METHODS

A. Data Collection

1) *Participants*: Participants consisted of this work's authors and volunteers at three different gyms. There were 13 participants in total (9 male, 4 female). All participants were about 18-25 years old and varied in weight, height, strength, and resistance training experience.

2) *Equipment*: Participant preacher curls were recorded using a custom-made recording device, consisting of a Raspberry Pi 5 and the Adafruit MPU-6050 6-axis inertial measurement unit (IMU), capturing both accelerometer and gyroscope data. Although only IMU data was used here, the device, shown in Figure 1 was designed with the intention and capability of recording data from multiple sensors (such as heartbeat sensor and sEMG). The Raspberry Pi read data from the IMU at 100 Hz, and was connected to and accessed by a laptop. Three different preacher curl machines at the three locations were used for data collection: (TODO).

3) *Procedure*: During data collection, the IMU was taped to each participant's left wrist. The axes of the sensor were oriented as shown in Figure 2. Each participant was instructed to perform sets of preacher curls with consistent form, taking each set to momentary muscular failure. Participants were free to select the weight they lifted and decide how many sets they performed. Consequently, collected data varied in the amount of weight lifted, the number of repetitions per set, and how many sets were completed in a session. Participants also introduced natural variation in form and range of motion (e.g. locking out their arms at the bottom of the curl, duration of pause at the bottom, height of the concentric phase). Only

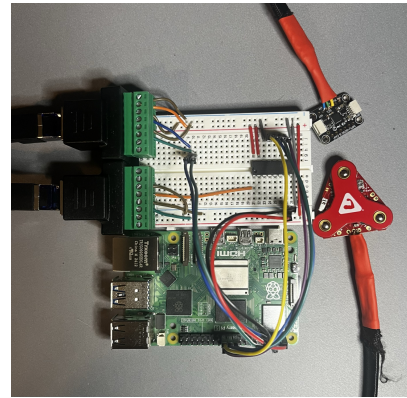


Fig. 1. **Data collection device.** The Raspberry Pi (bottom) reads, processes, and stores data from the IMU (top right). The sEMG sensor (bottom right) was not used due to hardware issues. During data collection, the Pi was powered by a portable battery pack. A box was used to protect the Raspberry Pi from damage, and the IMU was taped to participants' wrists.

sets taken to failure with proper form were admitted to this work's dataset.

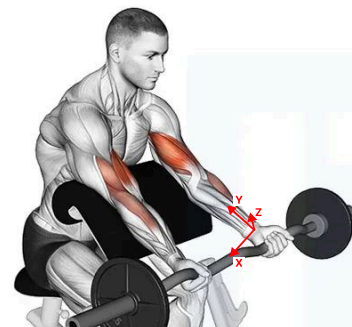


Fig. 2. **IMU orientation.** The IMU was taped to the participant's wrist with the axes oriented as shown. Image source [1].

During data collection, annotations were recorded for future labelling. This was facilitated by a custom Python server running on the Raspberry Pi. When prompted to start recording, the server read and stored data from the IMU in CSV format at approximately 100 Hz. The server broadcasts an interactive data collection web application for access via laptop over Wifi

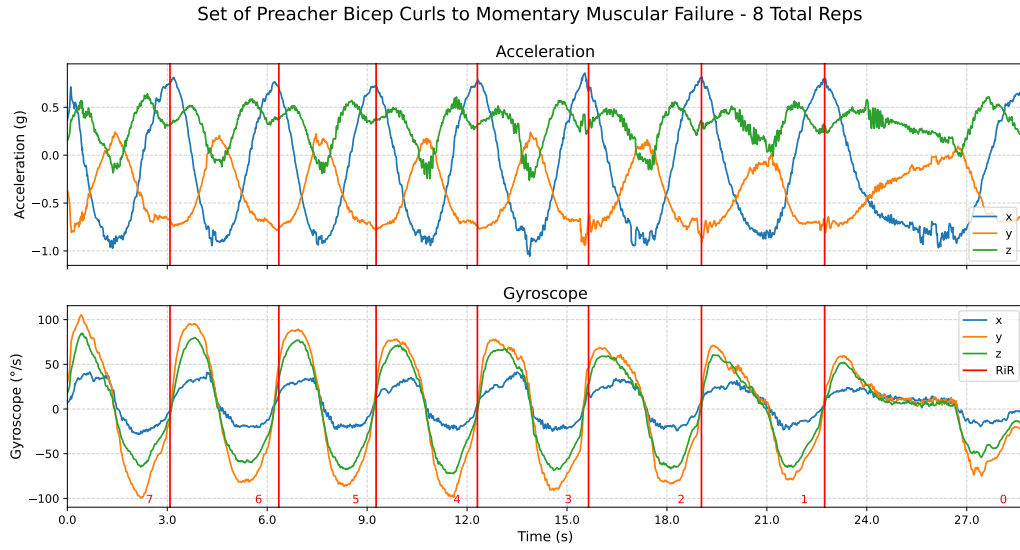


Fig. 3. **Full set of preacher curls.** The accelerometer (top) and gyroscope (bottom) data from a single set of preacher curls, with the end-of-rep annotations shown as red vertical lines. The set lasted 26.2 seconds and consisted of 8 repetitions to failure. Red numbers indicate the Repetitions in Reserve at each point during the set. The last rep, at which point the participant could no longer lift the weight, is a rep taken to failure, with an RiR of 0.

or Ethernet. This data collection dashboard was used to enter participant metadata and location metadata, start the recording of IMU data, manually record timestamp markers to annotate the end of each repetition, and stop the recording. The timestamp markers were later used to generate data labels.

In total, 13 participants performed 68 sets of preacher curls, with an length of 36 seconds and 9.3 curl repetitions. This resulted in 40 minutes of data consisting of 631 total reps.

B. Data Processing

Several steps were taken to process the raw IMU data and prepare it for model training.

1) *Data Cleaning*: First, to resolve fluctuations in data sampling rate caused by the Raspberry Pi's CPU load, the 6-channel timeseries data was first interpolated to exactly 100 Hz. Next, the data was smoothed with a 15-data point (150 ms) moving average filter to remove signal noise. The data annotations were adjusted as well. Manual inspection of the timeseries signals was used to adjust any incorrect end-of-rep annotations to the true end of the rep. A full set of curls after cleaning, along with the data annotations, can be seen in Figure 3.

2) *Splitting & Windowing*: To facilitate model training, the collected data was split into train and validation splits. About 80% of the recorded preacher curl sets (53) were added to the train set. The remaining 20% of sets (15) were added to the validation split. Next, the data in each split was windowed into 256 data-point (2.56s) segments, with a stride of 2 data points (20 ms). Each window was labelled based on the end-rep-marker annotations, as will be explored. In total, about 90k training windows and 26k validation windows were generated.

Splitting data at the set level guarantees that no overlap exists between the data in the training and validation set,

ensuring that the model is not trained on data it will be evaluated on. Windows of 2.56 seconds were chosen to balance the receptive field of the model with real time capabilities. Windows of 2.56 ensure models had the context needed to capture an entire rep, which are usually less than 2.56 seconds. At the same time, the 2.56 second windows are small enough to allow for real time inference on edge devices. A small stride was chosen to maximize the number of training samples available to the model.

3) *Data Augmentation*: To simulate additional data, the training data was augmented with several techniques. First, training samples were randomly stretched and cropped in the time dimension by a factor of 1 to 1.5, simulating faster reps. Next, the amplitude of the training windows were randomly scaled by a factor of 0.6 to 1.4, simulating more or less forceful reps. Augmentations were randomly applied to the training windows on every retrieval.

C. Segmentation Model Design

As a first step toward achieving the goals of this work, a segmentation model was designed to detect the end of each rep. This model was designed to take in a window of 6-channel IMU data and output a binary classification of whether each input data point is the end of a rep. To train the model, each window was given 256 labels (one per data point) based on the end-rep-markers. The area (80ms) around any end-rep-markers occurring within a window was labelled with a positive label. The remaining data points of each window were labelled with negative labels. An example of the labelling for a single window is shown in Figure 4.

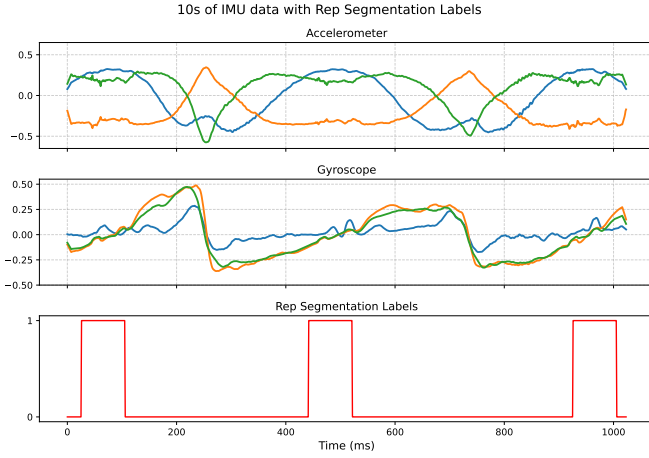


Fig. 4. **Segmentation labels.** A single window of 1024 data points. A larger window size is chosen here for demonstration purposes. Shown from top to bottom are the acceleration data, gyroscope data, and segmentation labels. The segmentation labels are binary, with 1 indicating the end of a rep.

D. Classification Model Design

The next step in this work was to design a classification model to predict near-failure reps. This model was designed to classify a window of IMU data as either a near-failure rep ($RiR \leq 2$) or not. The model was trained on the same data as the segmentation model, but with a different set of labels. Each window was labelled with a binary label indicating whether the entire window was a near-failure rep or not. Windows with over 50% of their data points in a region of $RiR \leq 2$ (for instance, the region corresponding to the last three reps in Figure 3) were given a positive label. The remaining windows were labelled with a negative label.

E. Model Architectures

1) *Segmentation Model:* The foundation of the segmentation model is a 1D convolutional residual neural network (ResNet) [2]. The model consists of several strided 1D convolutional layers, each followed by a batch normalization layer and a ReLU activation function. Residual connections are included around each convolutional layer. Convolutional layers with a kernel size of 1 are project residual connection channels when necessary. A final global average pooling and linear classification layer produce 256 output confidences, one for each data point in the input window. The final model architecture, found by neural architecture search (NAS) as explored in the next section, consists of 2,971,648 parameters and is shown in Figure 5 [3].

2) *Classification Model:* The base of the classification is the same ResNet base of the segmentation model. By loading the weights of the previous model, previously determined high dimensional representations of the IMU data can be used for further learning. This model also consists of an LSTM component

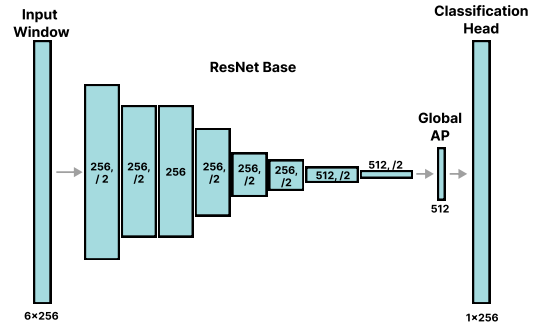


Fig. 5. **Segmentation model architecture.** Each block in the ResNet Base consists of a 1D Convolution layer with a kernel size of 3, followed by a batch normalization layer and a ReLU activation function. Residual connections are included around each block. Labels on each block represent the output channels and stride of each layer.

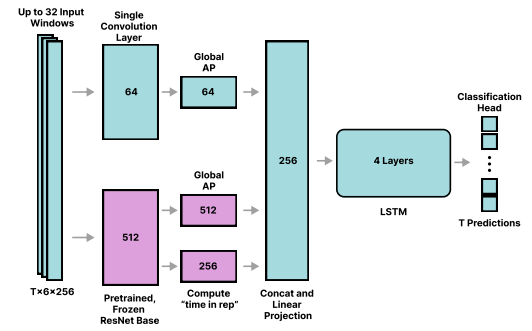


Fig. 6. **Classification model architecture.**

F. Neural Architecture Search

G. Model Training

H. Model Evaluation

I. Edge Deployment

describe steps taken to deploy to edge

III. RESULTS

Put any figures and tables here

A. Segmentation Model Performance

B. Classification Model Performance

C. Edge Deployment Performance

IV. DISCUSSION

REFERENCES

- [1] K. Kings, "Best preacher curl alternatives," accessed April 30, 2025. [Online]. Available: <https://www.kettlebellkings.com/blogs/default-blog/best-preacher-curl-alternatives>
- [2] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2016, pp. 770–778.
- [3] B. Zoph and Q. V. Le, "Neural architecture search with reinforcement learning," *arXiv preprint arXiv:1611.01578*, 2016.